

-  SiteSearch - 攻击性扫描系统
 -  项目定位
 -  系统架构（从 Recon → Exploit-ready）
 - 目标
 - 完整攻击链（仅到"准备利用"为止）
 -  阶段化实现目标
 - L0. 输入层（定义攻击面）
 - L1. 资产发现层（子域爆破 / 被动情报）
 - L2. 活性探测 & 指纹层
 - L3. 攻击面映射层
 - L4. Exploit-ready 输出层
 -  项目结构
 -  技术栈
 - 后端 (siteback)
 - 前端 (sitevue)
 - 核心工具集成
 -  安装部署
 - 环境要求
 - 快速开始
 - 1. 克隆项目
 - 2. 后端安装与启动
 - 3. 前端安装与启动
 -  API 接口
 - 任务管理接口
 - 清理服务接口
 - 扫描参数配置
 -  核心功能模块
 - 数据库管理
 - 任务调度系统
 - OneForAll 集成
 - 定时清理服务
 -  攻击者视角特性
 - 优先级目标识别
 - 技术栈反查与攻击路线匹配
 -  数据流架构
 - 输入处理流程
 - 输出数据结构化

- 🌟 未来规划
 - 短期目标 (L2层实现)
 - 中期目标 (L3层实现)
 - 长期目标 (L4层实现)
 - AI接入 (L5层实现)
- ⚠️ 法律与道德声明
- 💛 贡献指南
- 📞 联系方式



SiteSearch - 攻击性扫描系统

一个面向授权渗透测试场景的纯进攻性扫描框架，专注于攻击链前半段（Recon → Asset → Fingerprint → Vulnerability Surface Mapping）的自动化信息收集与深度探测。



项目定位

攻击者视角的信息收集 + 面向利用链准备的深度探测，不讨论防御、不讨论修复，也不涉及具体利用方法，但要把攻击链前半段做到极致、自动化，并保持技术链路完整、清晰、可落地。



系统架构（从 Recon → Exploit-ready）

目标

对授权范围内的目标生成可用于后续攻击操作的"利用前置情报"。

完整攻击链（仅到"准备利用"为止）

[资产输入]



[子域枚举 & 关联扩展]



[存活探测]
↓
[Web / 服务 指纹识别]
↓
[攻击面扫描 (Nuclei / 服务探测)]
↓
[路径/接口/后台收集]
↓
[技术栈反查 → 匹配潜在攻击路线]
↓
[生成 Exploit-ready 情报包]



阶段化实现目标

L0. 输入层（定义攻击面）

状态：✅ 已完成

输入：

- 主域 (target.com)
- 可选：IP 段、企业名、证书主体

技术实现：

- 基于 OneForAll 的目标输入与参数配置
- 任务管理与状态跟踪系统
- SQLite 数据库持久化存储

当前功能：

- ✅ 目标域名输入与验证
- ✅ 扫描参数配置（爆破、DNS、端口等）
- ✅ 异步任务执行与状态监控
- ✅ 实时进度跟踪与结果持久化

L1. 资产发现层（子域爆破 / 被动情报）

状态：✅ 已完成

核心思想：把 OneForAll 作为入口，但它不是终点，而是第一个源。

技术实现：

- OneForAll 集成与任务调度
- 多源子域名收集（证书透明度、搜索引擎、DNS记录等）
- 递归爆破与智能字典扩展

当前功能：

-  被动源收集（CT Logs、搜索引擎、DNSDB等）
-  主动爆破（内置字典 + 自定义字典）
-  结果格式统一化与标准化输出
-  实时进度监控与错误处理

L2. 活性探测 & 指纹层

状态：  开发中

****目标：** **从海量资产中筛出"可攻击目标"

计划实现：

- 多协议存活探测（HTTP/HTTPS + 常用端口）
- WAF/IPS 识别（基于响应 header/状态码/JARM 指纹）
- Web 指纹识别（服务端、前端框架、CDN/反代）
- 服务/端口指纹（masscan + nmap 集成）

技术细节：

- 自动跟随 301/302，记录跳转路径判断反向代理/CDN
- 技术堆栈识别（Spring/PHP/ASP.NET/Rails等）
- 特征性资源识别（swagger-ui、actuator、admin等）

L3. 攻击面映射层

状态：  规划中

****目标：** **生成"漏洞指示点列表"，供后续人工利用

计划实现：

- Nuclei 模板化漏洞扫描（攻击者真正有用的模板）

- 路径/目录收集（基于词典 + 技术栈推导）
- 登录点分析（csrf token、返回码模式、CAPTCHA检测）
- API 接口路径挖掘（JS静态分析、robots.txt抽取）

扫描分类：

- 信息泄露（API key、内部接口、.git/.env等）
- 弱点识别（弱登录面板、CVE版本指纹）
- 远程面板（Jenkins、Nexus、Shiro等）
- 云资产（S3/OSS存储桶、公开对象）

L4. Exploit-ready 输出层

状态： 📅 规划中

****目标：** **生成结构化的"攻击准备数据包"

输出格式：

```
{
  "subdomains": [...],
  "alive_services": [...],
  "web_fingerprints": {...},
  "service_fingerprints": {...},
  "attack_surfaces": [...],
  "potential_vulns": [...],
  "login_panels": [...],
  "api_endpoints": [...],
  "interesting_paths": [...],
  "tech_stack_map": {...},
  "priority_targets": [...]
```



项目结构

```
sitesearch/
├── OneForAll/      # L1层：子域名收集工具
├── siteback/       # 后端服务 (Flask)
│   ├── app/        # 应用主程序
│   │   └── main.py  # Flask 应用入口 + API路由
│   └── model/       # 核心代码模块
│       └── database.py # 数据库操作（任务管理）
```

```
├── handleofa.py # L1层: OneForAll任务处理
├── process.py   # 实时进度监控
├── result.py    # 结果文件处理
├── deleters.py  # 定时清理服务
└── sitevue/     # 前端 Vue 项目
    ├── src/
    │   ├── views/      # 页面组件
    │   │   └── Task.vue # 任务管理页面
    │   └── routes/     # 路由配置
    └── package.json    # 项目配置
```

技术栈

后端 (siteback)

- Python 3.7+ - 核心编程语言
- Flask 2.3.3 - Web框架与API服务
- SQLite - 轻量级数据库（任务状态存储）
- Threading - 异步任务处理
- OneForAll - 子域名收集引擎

前端 (sitevue)

- Vue.js 3 - 现代化前端框架
- TypeScript - 类型安全的JavaScript
- Vite - 快速构建工具
- 实时进度监控 - 动态任务状态展示

核心工具集成

- OneForAll - L1层资产发现
- 计划集成: masscan、nmap、Nuclei等攻击面扫描工具

安装部署

环境要求

- Python 3.7+
- Node.js 14+
- SQLite 3（内置，无需额外安装）

快速开始

1. 克隆项目

```
git clone [仓库地址]
cd sitesearch
```

2. 后端安装与启动

```
cd siteback
# 安装依赖
pip install -r requirements.txt
# 启动服务（自动启动定时清理服务）
python -m app.main
```

服务将在 <http://127.0.0.1:5000> 启动

3. 前端安装与启动

```
cd ../sitevue
# 安装依赖
npm install
# 开发模式运行
npm run dev
```

前端将在 <http://localhost:5173> 启动



API 接口

任务管理接口

- [POST /api/run](#) - 创建扫描任务

- GET /api/task - 获取任务列表/详情
- GET /api/process - 获取任务进度信息
- GET /api/result - 获取任务结果数据

清理服务接口

- GET /api/cleanup/status - 获取清理服务状态
- POST /api/cleanup/start - 启动清理服务
- POST /api/cleanup/stop - 停止清理服务
- POST /api/cleanup/manual - 手动执行清理

扫描参数配置

支持完整的 OneForAll 参数：

- target - 目标域名（必需）
- brute - 爆破模块开关
- dns - DNS查询模块
- req - HTTP请求模块
- port - 端口扫描
- 等更多高级参数



核心功能模块

数据库管理

- SQLite 数据库操作
- 线程安全的数据库连接
- 事务管理与错误处理
- 结果持久化存储

任务调度系统

- 异步任务执行
- 状态监控与进度跟踪

- 多线程任务处理
- 错误恢复机制

OneForAll 集成

- 命令行参数解析
- 结果文件处理
- 输出格式转换 (CSV/JSON/TXT)
- 实时进度监控

定时清理服务

- 24小时文件保留策略
- 自动清理过期扫描结果
- 手动清理接口支持
- 服务状态监控

攻击者视角特性

优先级目标识别

系统自动生成 **priority_targets** 优先级列表：

1. 有登录面板的管理端 - 高价值目标
2. 老旧技术栈 - 已知漏洞概率高
3. 云存储暴露 - 敏感信息泄露风险
4. 大量API接口 - 攻击面广泛
5. 未经保护的服务端口 - Redis/Mongo/MinIO等
6. 有CVE指纹的中间件 - 已知漏洞利用
7. 开放调试/监控端点 - actuator、debugger等

技术栈反查与攻击路线匹配

- 自动识别技术栈版本
- 匹配潜在攻击路线 (如 JDK → deserial, Spring → RCE surface)

- 生成攻击建议与优先级排序



数据流架构

输入处理流程

用户输入 → 参数验证 → 任务创建 → 异步执行 → 进度监控 → 结果收集 → 数据标准化

输出数据结构化

所有扫描结果统一为标准化格式，便于后续攻击面分析：

- 子域名信息（来源、时间戳、关联资产）
- 存活服务详情（端口、协议、指纹）
- Web指纹数据（框架、版本、特征）
- 攻击面映射（漏洞指示点、风险等级）



未来规划

短期目标（L2层实现）

- ☐ 集成 masscan 进行端口扫描
- ☐ 实现 Web 指纹识别系统
- ☐ 添加 WAF/CDN 识别功能
- ☐ 构建技术栈分析引擎

中期目标（L3层实现）

- ☐ Nuclei 模板化扫描集成
- ☐ 路径爆破与接口发现
- ☐ 登录点自动化分析
- ☐ 云资产识别与扫描

长期目标（L4层实现）

- ☐ 攻击面优先级算法
- ☐ 利用链建议生成
- ☐ 报告自动化生成
- ☐ 分布式扫描架构

AI接入（L5层实现）

- ☐ 基于大模型的攻击面分析
- ☐ 攻击利用链推荐系统
- ☐ 漏洞利用建议生成及报告
- ☐ 网络拓扑结构绘制



法律与道德声明

重要提醒：

1. 本工具仅限在授权范围内使用
2. 禁止用于非法渗透测试活动
3. 使用者需对自身行为承担法律责任
4. 建议在测试环境中先进行小规模验证



贡献指南

欢迎安全研究人员和开发者提交 Issue 和 Pull Request 来帮助改进本项目。



联系方式

如有问题或建议，请通过项目 Issue 系统联系。

****项目愿景：****构建一个真正从攻击者视角出发，专注于攻击链前半段自动化的专业扫描框架，为安全研究人员提供高效、准确的情报收集能力。